



GymTrack

Documentación técnica y funcional completa

Proyecto final de Tecnología Web Aplicada · 2026

Leandro González · Santiago Cáceres · Máximo Díaz · Emilio Escobar · Magdalena Belmonti · Hiliana
Pereira

Índice

1.  GymTrack — Mapa de Contenidos
2. Metodología y Alcance
3. Casos de Uso
4. Análisis y Requerimientos
5. Arquitectura del Sistema
6. Modelo de Base de Datos
7. Sistema de Diseño
8. Decisiones Técnicas
9. Bitácora del Proyecto
10. Estrategia de Testing
11. Seguridad de la API
12. Manual de Despliegue
13. Librerías Utilizadas
14. Implicancias Éticas
15. Conclusiones del Proyecto
16. Guion de Presentación
17. Guía de Defensa — Backend
18. Guía de Defensa — Frontend
19. Guía de Defensa — Base de Datos
20. Preguntas Frecuentes de Defensa



GymTrack — Mapa de Contenidos

Vault de documentación técnica y funcional de **GymTrack**, plataforma web para descubrir gimnasios y gestionar su operación diaria. Pensado para abrirse con [Obsidian](#): todas las notas están enlazadas entre sí con `[[wikilinks]]`, así que basta con hacer clic para saltar de una a otra.

Stack: Vue 3 + Pinia + Vue Router (frontend), PHP 8.2 + PDO + MySQL 8 (backend), Docker Compose (infraestructura). Ver [Librerías Utilizadas](#) para el detalle exacto de versiones.

Cómo está organizado

Carpeta	Contenido
00-Defensa/	Guías para la presentación oral del proyecto
01-Planificacion/	Metodología, alcance y casos de uso
02-Analisis/	Requerimientos funcionales y no funcionales
03-Arquitectura/	Cómo está armado el sistema, capa por capa
04-Disenio/	Sistema de diseño visual e interacción
05-Desarrollo/	Decisiones técnicas tomadas durante el desarrollo
06-Bitacoras/	Línea de tiempo real del proyecto, con fechas de commits
07-Testing/	Qué se prueba, cómo, y qué falta
Despliegue/	Cómo levantar el proyecto local y en producción
Notas sueltas en la raíz	Seguridad, ética, conclusiones

Recorrido sugerido

Si venís a entender el proyecto desde cero

1. [Metodología y Alcance](#) — qué se construyó y qué se dejó afuera a propósito
2. [Casos de Uso](#) — quién usa la app y para qué
3. [Análisis y Requerimientos](#) — requerimientos funcionales y no funcionales
4. [Arquitectura del Sistema](#) — cómo está armado por dentro
5. [Modelo de Base de Datos](#) — las 50 tablas reales y cómo se relacionan

Si venís a prepararte para la defensa

1. [Guion de Presentación](#)
2. [Guía de Defensa - Backend](#) · [Guía de Defensa - Frontend](#) · [Guía de Defensa - Base de Datos](#)
3. [Preguntas Frecuentes de Defensa](#)

Si venís a instalar o desplegar el proyecto

1. [Manual de Despliegue](#)
2. [Librerías Utilizadas](#)

Si te interesa la seguridad

1. [Seguridad de la API](#)
 2. [Implicancias Éticas](#)
-



Todas las notas

Planificación y análisis

- [Metodología y Alcance](#)
- [Casos de Uso](#)
- [Análisis y Requerimientos](#)

Arquitectura y diseño

- [Arquitectura del Sistema](#)
- [Modelo de Base de Datos](#)
- [Sistema de Diseño](#)

Desarrollo

- [Decisiones Técnicas](#)
- [Bitácora del Proyecto](#)

Calidad

- [Estrategia de Testing](#)
- [Seguridad de la API](#)

Operación

- [Manual de Despliegue](#)
- [Librerías Utilizadas](#)

Cierre del proyecto

- [Implicancias Éticas](#)
- [Conclusiones del Proyecto](#)

Defensa

- [Guion de Presentación](#)
 - [Guía de Defensa - Backend](#)
 - [Guía de Defensa - Frontend](#)
 - [Guía de Defensa - Base de Datos](#)
 - [Preguntas Frecuentes de Defensa](#)
-

Sobre esta documentación

Todo lo que describe **código, tablas, comandos y flujos** en este vault está verificado contra el repositorio real — no hay funciones inventadas ni métricas de uso fabricadas. GymTrack es un proyecto académico con un dataset de demostración, no tiene usuarios de producción reales, así que no vas a encontrar acá frases como "usado por 3 gimnasios reales" o "200 socios gestionados": esas cifras no existen y no correspondía inventarlas.

Las secciones de **reflexión personal** ([Conclusiones del Proyecto](#)) quedaron como plantilla para que cada integrante del equipo las complete con su propia experiencia — no es algo que se pueda escribir en nombre de otra persona.

Equipo: Leandro González · Santiago Cáceres · Máximo Díaz · Emilio Escobar · Magdalena Belmonti · Hiliana Pereira **Materia:** Tecnología Web Aplicada · 2026

Metodología y Alcance

← [Mapa de Contenidos](#)

Metodología de desarrollo

El proyecto se construyó por **entregas funcionales incrementales**, no por sprints de tiempo fijo: cada etapa cierra un dominio completo (autenticación, catálogo público, administración, operación multi-gimnasio, agenda, pagos) antes de pasar a la siguiente. Se puede seguir la evolución real en [Bitácora del Proyecto](#), reconstruida directamente desde el historial de Git del repositorio.

El propio proyecto numera explícitamente algunas de sus etapas en el nombre de las migraciones de base de datos (`phase1` , `phase3` , `phase4` , `phase5` , `phase9` , `phase10` — sin `phase2` , `phase6` , `phase7` ni `phase8` como archivos propios, esos tramos de trabajo existen igual pero no quedaron numerados en ningún archivo real):

- **Fase 1** (`001_phase1_integrity.sql`) — integridad base de identidad.
- **Fase 3** (`002_phase3_roles_demo_context.sql` , `003_phase3_identity_security.sql`) — roles, contexto multi-gimnasio, dataset demo y seguridad de identidad.
- **Fase 4** (`004_phase4_administration.sql`) — shell de administración.
- **Fase 5** (`005_phase5_tenant_operations.sql`) — operación multi-gimnasio, con permisos independientes por gimnasio.
- (sin número de fase en el archivo, pero con commit propio) agenda de clases, reservas y asistencia (`006_class_schedule_booking_attendance.sql`); pagos, finanzas y reportes (`007_payments_finance_reports.sql`); geolocalización y prioridad de gimnasios en el mapa.
- **Fase 9** (`008_phase9_calendar_promotions_notifications.sql`) — calendario, promociones y notificaciones.
- **Fase 10** (`009_phase10_member_experience.sql`) — experiencia del socio.
- (sin número de fase) país, estados operativos y geocodificación de gimnasios (`010_gimnasios_mapa_sincronizado.sql`).
- **Cierre** — endurecimiento de seguridad, renombrado del código a español y pulido visual (ver [Decisiones Técnicas](#)).

Framework técnico

Capa	Tecnología
Frontend	Vue 3 (Composition API), Pinia, Vue Router, Axios
Backend	PHP 8.2 con autoloader por convención (sin framework), PDO
Base de datos	MySQL 8
Mapas	Leaflet + OpenStreetMap

Capa	Tecnología
Infraestructura	Docker y Docker Compose (dev y producción por separado)
Pruebas	Vitest, Playwright, scripts de contratos de API en shell y PHP

A diferencia de otros proyectos de la misma materia, el backend de GymTrack **no usa un framework como Laravel o Symfony**: es PHP 8.2 con tipado estricto (`declare(strict_types=1)`), un autoloader propio (`spl_autoload_register` en `backend/index.php`) que resuelve clases por convención de carpeta (`controllers/` , `models/` , `services/` , `middleware/`), y una tabla de rutas explícita en `backend/routes/api.php` . Esa decisión está justificada en [Arquitectura del Sistema](#).

Control de versiones

Git, con mensajes de commit en inglés siguiendo el estilo `tipo: descripción` (`feat:` , `fix:` , `chore:` , `docs:` , `release:`). No se usó un flujo de ramas tipo GitFlow: el desarrollo avanzó sobre una rama principal por fase, con "baselines aprobadas" marcadas explícitamente en los mensajes de commit antes de avanzar a la siguiente fase.

Licencia

El repositorio no incluye actualmente un archivo `LICENSE` explícito. Al ser un proyecto académico de cierre de materia, el código se comparte con fines educativos entre el equipo y la institución; si el equipo decide publicarlo como portfolio público, corresponde agregar una licencia explícita (MIT o similar) antes de esa publicación.

Alcance: qué está construido

Ver el detalle completo en [Casos de Uso](#). Resumen por área:

- ✓ Autenticación completa (registro, verificación de correo, login, recuperación y cambio de contraseña, sesiones revocables, CSRF)
- ✓ Catálogo público con mapa interactivo, filtros y geolocalización opcional
- ✓ Administración multi-gimnasio con permisos independientes por gimnasio
- ✓ Gestión de socios, empleados, entrenadores e invitaciones de un solo uso
- ✓ Planes de membresía versionados, altas y renovaciones
- ✓ Agenda de clases recurrentes con cupos, lista de espera y asistencia
- ✓ Pagos manuales auditados + integración con Mercado Pago (checkout y webhooks firmados)
- ✓ Finanzas y reportes (Excel y PDF) generados desde MySQL
- ✓ Promociones con programación, pausa y métricas
- ✓ Notificaciones internas y por correo, con consentimiento y preferencias
- ✓ Carné digital del socio con QR firmado
- ✓ Área personal del socio (progreso, mediciones, favoritos, historial)

Alcance: qué se dejó explícitamente afuera

Estas funciones existen como **integraciones desactivadas por defecto** (feature flags), no como promesas incumplidas — el sistema las declara así mismo y no bloquea ningún flujo estable por su ausencia:

Función	Estado real
WhatsApp	Adaptador que falla de forma segura; falta un proveedor de envío real configurado
Sincronización automática con Google Calendar	Funciona el enlace manual + descarga de ICS; falta OAuth bidireccional
Analítica avanzada	Las métricas financieras y operativas reales funcionan; no hay predicciones
Recomendaciones personalizadas	Favoritos y preferencias funcionan; no hay motor de recomendación automatizado

Esta decisión de diseño (declarar explícitamente lo que falta, en vez de simularlo) es intencional: evita que una demo muestre una función que en producción real no haría nada.

Ver también: [Casos de Uso](#) · [Análisis y Requerimientos](#) · [Arquitectura del Sistema](#)

Casos de Uso

← [Mapa de Contenidos](#) · Ver también [Análisis y Requerimientos](#)

Los casos de uso están agrupados por rol. GymTrack tiene cuatro roles: **socio**, **empleado**, **dueño** y **administrador general**, más un visitante anónimo (sin cuenta) para la experiencia pública.

Visitante anónimo (sin cuenta)

- **UC-01:** Explorar el catálogo público de gimnasios verificados.
- **UC-02:** Buscar y filtrar por nombre, ciudad, categoría, servicio y estado.
- **UC-03:** Ver gimnasios en un mapa interactivo (Leaflet + OpenStreetMap), con marcadores agrupados.
- **UC-04:** Pedir la ubicación del navegador (con consentimiento) para ordenar por cercanía, sin que el servidor guarde esa ubicación.
- **UC-05:** Ver la ficha pública de un gimnasio: sedes, horarios, contacto y planes.
- **UC-06:** Registrarse como socio.
- **UC-07:** Enviar una solicitud de alta como dueño de gimnasio (crea una cuenta sin permisos operativos hasta que un administrador la revise).

Identidad y acceso (todos los roles autenticados)

- **UC-08:** Iniciar sesión con cookie `HttpOnly`.
- **UC-09:** Verificar el correo electrónico mediante enlace de un solo uso.
- **UC-10:** Recuperar la contraseña.
- **UC-11:** Cambiar la contraseña estando autenticado.
- **UC-12:** Ver y revocar sesiones activas.
- **UC-13:** Cerrar sesión.

Socio

- **UC-14:** Ver el panel personal (próxima clase, membresía, pagos pendientes, progreso mensual).
- **UC-15:** Consultar y reservar clases en la agenda, con lista de espera si no hay cupo.
- **UC-16:** Cancelar una reserva propia.
- **UC-17:** Agregar una clase reservada a Google Calendar (enlace manual) o descargarla como archivo `.ics`.
- **UC-18:** Ver el historial de pagos y membresías propias.
- **UC-19:** Ver y editar el perfil (datos personales, foto).
- **UC-20:** Configurar preferencias de notificación y objetivo mensual de asistencia.
- **UC-21:** Registrar mediciones opcionales (peso, altura) y ver la tendencia en un gráfico.
- **UC-22:** Ver el IMC calculado, siempre acompañado de una advertencia de que es orientativo, no diagnóstico.
- **UC-23:** Marcar gimnasios y actividades como favoritos.

- **UC-24:** Ver el carné digital propio, con un código QR firmado y de corta duración.
- **UC-25:** Cambiar de gimnasio activo si tiene más de una asociación.

Empleado / Dueño (administración de un gimnasio)

- **UC-26:** Seleccionar el gimnasio activo (contexto administrativo).
- **UC-27:** Gestionar socios: alta, edición, historial.
- **UC-28:** Gestionar empleados y entrenadores mediante invitaciones de un solo uso.
- **UC-29:** Crear y versionar planes de membresía (UYU o USD).
- **UC-30:** Dar de alta, renovar y cambiar el estado de membresías.
- **UC-31:** Crear y editar clases recurrentes, con sedes, entrenadores y cupos.
- **UC-32:** Gestionar reservas, lista de espera (con promoción automática al liberarse un cupo) y asistencia.
- **UC-33:** Registrar pagos manuales, con estados (pendiente, aprobado, rechazado, vencido, reembolsado, cancelado).
- **UC-34:** Consultar finanzas: ingresos mensuales, comparación entre períodos, deuda.
- **UC-35:** Generar y descargar reportes en Excel o PDF.
- **UC-36:** Crear y programar promociones (con audiencia, pausa, finalización e imagen).
- **UC-37:** Ver métricas reales de una promoción.
- **UC-38:** Configurar la información del gimnasio y sus sedes (dirección, geocodificación en el mapa).
- **UC-39:** Verificar el carné digital de un socio en el mostrador (token firmado, corta duración).
- **UC-40:** Archivar lógicamente un gimnasio sin destruir su historial.

Administrador general

- Todos los casos de uso de Empleado/Dueño, sobre **cualquier gimnasio**, en **modo soporte auditado**: entrar al contexto de un gimnasio ajeno exige escribir un motivo, que queda registrado en la auditoría (`audit_logs`).
- **UC-41:** Supervisar la plataforma completa (todos los gimnasios).
- **UC-42:** Consultar el historial operativo y la trazabilidad administrativa de cualquier gimnasio.

Casos de uso transversales (sistema)

- **UC-43:** Procesar un webhook firmado de Mercado Pago y actualizar el estado de un pago solo después de confirmar directamente con el proveedor (nunca confiando en la redirección de éxito del navegador).
- **UC-44:** Enviar notificaciones internas y por correo, respetando consentimiento y preferencias, con reintentos y prevención de duplicados.
- **UC-45:** Frenar automáticamente intentos de login repetidos, por cuenta y por IP (ver [Seguridad de la API](#)).
- **UC-46:** Poblar (o limpiar) un dataset de demostración completo, identificado como tal en cada fila, sin mezclarse con datos reales.

Ver también: [Metodología y Alcance](#) · [Análisis y Requerimientos](#) · [Arquitectura del Sistema](#)

Análisis y Requerimientos

← [Mapa de Contenidos](#) · Ver también [Casos de Uso](#)

Requerimientos funcionales

Agrupados por módulo, con referencia al caso de uso correspondiente en [Casos de Uso](#).

Módulo	Requerimiento	Caso de uso
Identidad	El sistema debe permitir registro, verificación de correo, login, recuperación y cambio de contraseña	UC-06 a UC-13
Identidad	El sistema debe permitir revocar sesiones activas individualmente	UC-12
Catálogo	El sistema debe mostrar solo gimnasios verificados, con estado activo o cierre temporal	UC-01 a UC-05
Mapa	El sistema debe calcular distancias en el navegador, sin enviar la ubicación exacta al servidor	UC-04
Administración	El sistema debe aislar los datos de cada gimnasio entre sí (multi-tenant)	UC-26
Administración	El sistema debe permitir permisos independientes por gimnasio para un mismo usuario	UC-26 a UC-40
Agenda	El sistema debe evitar sobrecupo en una clase y promover automáticamente desde la lista de espera	UC-32
Pagos	El sistema debe registrar pagos manuales auditados y procesar pagos electrónicos solo mediante webhook verificado	UC-33, UC-43
Notificaciones	El sistema debe respetar el consentimiento y las preferencias del destinatario antes de enviar cualquier notificación	UC-44
Socio	El sistema debe presentar el IMC como referencia, nunca como diagnóstico	UC-22
Socio	El sistema debe generar un carné digital verificable con vencimiento corto	UC-24, UC-39

Requerimientos no funcionales

Seguridad

Ver el detalle completo en [Seguridad de la API](#). En resumen: consultas parametrizadas (PDO), contraseñas con `password_hash`, cookies `HttpOnly` / `SameSite` / `Secure`, protección CSRF, rate limiting por cuenta y por IP, tokens de un solo uso hasheados, validación de archivos subidos fuera del webroot, firma HMAC en webhooks de pago.

Disponibilidad y despliegue

El despliegue de producción usa imágenes separadas para backend y frontend, contenedores de solo lectura, un servicio `schema-check` que rechaza el arranque si falta una migración de base de datos, y un script de preflight (`scripts/production-preflight.sh`) que bloquea el despliegue si detecta configuración insegura (ver [Manual de Despliegue](#)).

Rendimiento

- La geocodificación de direcciones usa una caché local (`geocodificacion_cache`) y limita las consultas al proveedor externo a una por segundo.
- Los reportes financieros se generan de forma asíncrona con un estado de progreso, no de forma bloqueante.

Usabilidad y accesibilidad

- Diseño mobile-first: navegación adaptada desde 360 px de ancho, con panel inferior de navegación en móvil y bottom sheet de tres posiciones para el mapa.
- Componentes de UI propios (ver [Sistema de Diseño](#)) con foco visible, `aria-label` en botones de solo ícono, y manejo de foco (`focus trap`) en diálogos y cajones.
- Pruebas de accesibilidad automatizadas con `axe-core` en la suite de Playwright (`frontend/e2e/`).

Mantenibilidad

- Nomenclatura de archivos y clases en español para todo lo que es vocabulario de negocio (`SocioController` , `PagoRepository` , `AgendaController`); los sufijos de arquitectura (`Controller` , `Service` , `Repository` , `View`) se mantienen en inglés por ser terminología técnica estándar. Ver el razonamiento completo en [Decisiones Técnicas](#).
- Migraciones de base de datos versionadas, cada una con su reversión (`.down.sql`) — con la excepción documentada de las dos migraciones fundacionales, ver [Modelo de Base de Datos](#).

Restricciones del proyecto

- **Sin framework backend:** se optó por PHP puro con un autoloader propio en vez de Laravel/Symfony, una decisión curricular explícita (ver [Arquitectura del Sistema](#)).
- **Datos ficticios claramente identificados:** cualquier registro de demostración lleva una marca (`is_demo` , `demo_dataset_id`) y puede eliminarse sin tocar datos reales.
- **Sin diagnóstico médico:** cualquier cálculo relacionado a salud (IMC) se presenta únicamente como referencia.
- **Sin confirmar acciones que no ocurrieron:** ninguna función beta (WhatsApp, sincronización automática de calendario, recomendaciones) simula éxito si el proveedor real no está configurado.

Stakeholders

Rol	Interés principal
Socio	Encontrar un gimnasio, reservar clases, ver su progreso y pagar sin fricción
Dueño de gimnasio	Gestionar su operación diaria (socios, clases, pagos, finanzas) sin depender de planillas sueltas
Empleado / entrenador	Operar el día a día (asistencia, reservas) con los permisos que le correspondan
Administrador general	Supervisar la plataforma completa y dar soporte auditado
Equipo docente / tribunal	Evaluar el proyecto como cierre de la materia Tecnología Web Aplicada

Ver también: [Metodología y Alcance](#) · [Casos de Uso](#) · [Seguridad de la API](#) · [Decisiones Técnicas](#)

Arquitectura del Sistema

← [Mapa de Contenidos](#) · Ver también [Modelo de Base de Datos](#) · [Seguridad de la API](#)

Visión general



Tres contenedores Docker (`db` , `backend` , `frontend`) sobre una red interna compartida. El frontend nunca habla directo con MySQL; todo pasa por la API PHP. Ver [Manual de Despliegue](#) para el detalle de `docker-compose.yml` .

Backend: PHP sin framework

Decisión curricular deliberada: en vez de Laravel o Symfony, el backend es PHP 8.2 con tipado estricto y una estructura de capas hecha a mano.

Autoload por convención

`backend/index.php` registra un único autoloader:

```
spl_autoload_register(function (string $clase) {
    $carpetas = ['config', 'controllers', 'models', 'middleware', 'services'];
    foreach ($carpetas as $carpeta) {
        $ruta = __DIR__ . "/{$carpeta}/{ $clase }.php";
        if (file_exists($ruta)) { require_once $ruta; return; }
    }
});
```

Esto impone una regla simple pero estricta: **el nombre del archivo tiene que coincidir exactamente con el nombre de la clase**. Es la razón por la que, cuando se renombró el código de inglés a español (ver [Decisiones Técnicas](#)), cada archivo `.php` se renombró *junto con* la clase que declara, nunca por separado.

Capas

Capa	Carpeta	Responsabilidad
Rutas	<code>routes/api.php</code>	Mapea método + path HTTP a un método de controlador

Capa	Carpeta	Responsabilidad
Middleware	<code>middleware/</code>	<code>AuthMiddleware</code> — sesión, CSRF, roles y contexto de gimnasio
Controladores	<code>controllers/</code>	Validan la petición HTTP y orquestan servicios/modelos
Modelos / Repositorios	<code>models/</code>	Consultas SQL vía PDO, siempre con aislamiento por <code>gimnasio_id</code>
Servicios	<code>services/</code>	Lógica transversal: sesiones, tokens, pagos, notificaciones, seguridad

Controladores reales de `backend/controllers/` (verificado directamente contra el filesystem):

`AuthController` · `SocioController` · `AdminController` · `AdminApiController` · `AdminManagementController` · `AdminFileController` · `AgendaController` · `ClasesController` · `MembresiaController` · `PagoController` · `PromocionController` · `NotificacionController` · `ReporteController` · `CalendarioController` · `GeocodificacionController` · `MeController` · `PerfilController` · `ReservasController` · `PublicGymController` · `SystemController` .

`Clases` y `Membresia` ya estaban en español desde el inicio del proyecto; el resto de los nombres de dominio (`Socio` , `Pago` , `Promocion` , `Notificacion` , `Reporte` , `Agenda` , `Calendario` , `Geocodificacion`) se tradujeron después — ver [Decisiones Técnicas](#). `Admin` , `Auth` , `Public` , `System` y `Me` se dejaron en inglés a propósito: son prefijos de arquitectura/alcance, no vocabulario del negocio.

Autorización y multi-tenant

Cada gimnasio es un *tenant* aislado. La tabla `usuario_gimnasio_roles` conecta un usuario con un gimnasio y un rol específico para ESE gimnasio — una misma persona puede ser dueño de un gimnasio y empleado de otro. Casi toda consulta de `models/` filtra explícitamente por `gimnasio_id` (y por `demo_dataset_id` cuando corresponde), incluso cuando el controlador ya validó el contexto — una defensa en profundidad contra errores de un futuro cambio en el controlador.

`AuthorizationService` calcula los **permisos efectivos** de un usuario para el gimnasio activo (`usuario_gimnasio_permisos`), no roles fijos: un empleado puede tener permisos distintos en cada gimnasio donde trabaja.

Un administrador general que entra al contexto de un gimnasio ajeno activa el **modo soporte**: la acción exige un motivo por escrito, que queda en `audit_logs` (verificado en vivo: intentar seleccionar un gimnasio sin `reason` devuelve `422 support_reason_required`).

Frontend: Vue 3 + Pinia

Estructura

```

frontend/src/
├─ views/           # una vista por ruta (PanelView, GimnasiosView, DetalleGimnasio)
├─ components/      # admin/, member/, public/, notifications/, ui/
├─ stores/          # un store de Pinia por dominio (socio, pagos, agenda, admin...)
├─ services/api.js   # cliente Axios único, con interceptor de errores
├─ router/index.js   # rutas + guards de navegación
└─ styles/tokens.css # design tokens

```

Sesión y CSRF, del lado del cliente

El frontend **nunca guarda el token de sesión**: la cookie `gymtrack_session` es `HttpOnly`, invisible para JavaScript. El store `auth.js` guarda en memoria únicamente el `csrf_token` que devuelve el backend en cada login, y lo reenvía en el header `X-CSRF-Token` de cada petición que muta estado (ver [Seguridad de la API](#)).

Componentes reutilizables

Un design system propio en `components/ui/` (`AppButton`, `AppCard`, `AppDialog`, `AppDrawer`, `AppInput`, `AppSelect`, etc.) evita reinventar estilos en cada vista. El detalle visual está en [Sistema de Diseño](#).

Infraestructura

Servicio	Imagen	Rol
db	mysql:8.0	Base de datos, con las migraciones montadas en <code>docker-entrypoint-initdb.d</code>
backend	PHP 8.2 + Apache (<code>Dockerfile.prod</code> en producción)	API REST
frontend	Node 20 en dev / Nginx en producción	SPA de Vue

En producción, un cuarto servicio (`schema-check`) verifica que todas las migraciones requeridas estén aplicadas **antes** de permitir que arranque el backend — si falta una, el despliegue se detiene en vez de arrancar con un esquema incompleto.

Por qué esta arquitectura y no otra

- **PHP sin framework**: permite mostrar en la defensa un entendimiento explícito de autenticación, sesiones, CSRF y autorización sin que un framework los resuelva "por debajo" — es más trabajo, pero es más demostrable.
- **Multi-tenant por `gimnasio_id` en vez de bases de datos separadas por gimnasio**: más simple de operar (una sola base, una sola migración), y suficiente porque el aislamiento se hace a nivel de fila (row-level), verificado en cada consulta.

- **Sin ORM:** PDO directo con consultas preparadas — más verboso, pero sin la capa de "magia" de un ORM que puede esconder una consulta N+1 o una fuga de datos entre tenants.
-

Ver también: [Modelo de Base de Datos](#) · [Seguridad de la API](#) · [Decisiones Técnicas](#) · [Sistema de Diseño](#)

Modelo de Base de Datos

← [Mapa de Contenidos](#) · Ver también [Arquitectura del Sistema](#)

MySQL 8, **50 tablas reales** (contadas directamente con `grep "CREATE TABLE"`

`database/migrations/*.sql` `database/gymtrack_database.sql`, resultado verificado con `wc -l`),
construidas de forma incremental en 10 migraciones versionadas más el esquema base.

Migraciones, en orden (verificado contra `database/migrations/`)

Archivo	Reversión (<code>.down.sql</code>)	Qué agrega
<code>gymtrack_database.sql</code> (bootstrap)	—	Esquema mínimo inicial (de un solo gimnasio)
<code>001_phase1_integrity.sql</code>	✗ No tiene	Integridad base de identidad
<code>002_phase3_roles_demo_context.sql</code>	✗ No tiene	Roles, contexto multi-gimnasio, dataset demo
<code>003_phase3_identity_security.sql</code>	✓	Seguridad de identidad: tokens, rate limiting, consentimientos
<code>004_phase4_administration.sql</code>	✓	Administración
<code>005_phase5_tenant_operations.sql</code>	✓	Operación multi-tenant
<code>006_class_schedule_booking_attendance.sql</code>	✓	Agenda, reservas, asistencia
<code>007_payments_finance_reports.sql</code>	✓	Pagos, finanzas, reportes
<code>008_phase9_calendar_promotions_notifications.sql</code>	✓	Calendario, promociones, notificaciones
<code>009_phase10_member_experience.sql</code>	✓	Experiencia del socio
<code>010_gimnasios_mapa_sincronizado.sql</code>	✓	País, estados operativos y geocodificación de gimnasios

⚠ Las migraciones 001 y 002 son las únicas sin `.down.sql` — son las dos más fundacionales (crean la base multi-tenant completa) y revertirlas de forma segura requeriría más cuidado del que da un `DROP` genérico. El resto (003 a 010) sí tiene su reversión.

Las 50 tablas, agrupadas (lista completa, sin omitir ninguna)

Identidad y seguridad

`usuarios` · `roles` · `permisos` · `rol_permisos` · `usuario_gimnasio_roles` ·
`usuario_gimnasio_permisos` · `user_sessions` · `email_verification_tokens` ·
`password_reset_tokens` · `rate_limit_attempts` · `security_events` · `audit_logs` ·
`user_consent`s · `marketing_unsubscribe_tokens`

Gimnasios y catálogo

`gimnasios` · `gimnasio_sedes` · `geocodificacion_cache`

Personas dentro de un gimnasio

`empleado_perfiles` · `entrenador_perfiles` · `entrenador_gimnasios` ·
`invitaciones_gimnasio` · `owner_registration_requests`

Membresías

`planes_membresia` · `membresias` · `membresia_historial`

Agenda y asistencia

`clases` · `sesiones_clase` · `reservas` · `reserva_eventos` · `asistencias`

Pagos y finanzas

`pagos` · `pago_eventos` · `pago_reembolsos` · `exports`

Promociones y notificaciones

`promociones` · `promocion_gimnasios` · `notificaciones` · `notificacion_envios` ·
`notificacion_preferencias`

Experiencia del socio

`actividades` (catálogo estable de tipos de actividad por gimnasio, del que cuelgan los favoritos) ·
`socio_perfiles` · `socio_preferencias` · `socio_mediciones` · `socio_gimnasios_favoritos` ·
`socio_actividades_favoritas` · `calificaciones`

Datos de demostración y control interno

Convenciones reales del esquema

- **Aislamiento por tenant:** casi toda tabla operativa tiene `gimnasio_id`; las que además pueden ser demo tienen `is_demo` y/o `demo_dataset_id`.
- **Tokens nunca en texto plano:** los de verificación de correo y recuperación de contraseña se guardan hasheados (SHA-256), nunca en claro.
- **Rate limiting sin guardar el dato en claro:** `rate_limit_attempts` guarda `sujeto_hash` e `ip_hash` (HMAC), no el correo ni la IP reales — ver [Seguridad de la API](#).
- **Eventos, no solo estado final:** `pago_eventos` y `reserva_eventos` guardan el historial completo de cambios de estado, no solo el estado actual.
- **Borrado lógico, no físico:** un gimnasio se archiva (cambia de estado), no se elimina.

Ejemplo real: `rate_limit_attempts`

```
CREATE TABLE rate_limit_attempts (  
  id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,  
  accion VARCHAR(60) NOT NULL,  
  sujeto_hash CHAR(64) NOT NULL,  
  ip_hash CHAR(64) NOT NULL,  
  intentos INT UNSIGNED NOT NULL DEFAULT 0,  
  primer_intento_en DATETIME NOT NULL,  
  ultimo_intento_en DATETIME NOT NULL,  
  bloqueado_hasta DATETIME NULL,  
  PRIMARY KEY (id),  
  UNIQUE KEY uq_rate_limit_key (accion, sujeto_hash, ip_hash),  
  KEY idx_rate_limit_cleanup (ultimo_intento_en, bloqueado_hasta)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

Extraído tal cual de `003_phase3_identity_security.sql`. Ver el razonamiento del limitador en [Seguridad de la API](#).

Esquema base vs. esquema real

`database/gymtrack_database.sql` (el "bootstrap") por sí solo describe un esquema de un solo gimnasio, sin las columnas `gimnasio_id` que definen el modelo multi-tenant actual. Funciona correctamente porque las 10 migraciones se aplican siempre encima y son idempotentes — pero **leer solo ese archivo da una idea equivocada del esquema real**. Para entender el esquema actual hay que leer el bootstrap + las 10 migraciones en orden, o consultar directamente una base ya migrada.

Ver también: [Arquitectura del Sistema](#) · [Seguridad de la API](#) · [Manual de Despliegue](#)

Sistema de Diseño

← [Mapa de Contenidos](#) · Ver también [Análisis y Requerimientos](#)

GymTrack tiene un design system propio (no usa una librería de componentes externa como Vuetify o PrimeVue), definido en `frontend/src/styles/tokens.css` y aplicado a través de ~20 componentes reutilizables en `frontend/src/components/ui/`.

Identidad visual

- **Tema:** oscuro por defecto y único (`color-scheme: dark` fijo en `:root` — no hay toggle claro/oscuro).
- **Tipografía:** Manrope Variable, con `system-ui` como respaldo.
- **Color de acento:** azul (`#0969da`), usado para enlaces, elementos activos y foco.
- **Logo:** isotipo geométrico "G" en blanco/azul, en `frontend/src/assets/gymtrack-mark.svg` — el mismo archivo se usa en toda la app.

Paleta de tokens (extracto real de `tokens.css`)

```
--bg-canvas: #0b0e12;
--surface-1: #141a21;
--surface-2: #19212b;
--border-subtle: #27313d;
--text-primary: #f4f7f9;
--text-secondary: #aeb9c5;
--accent: #0969da;
--focus: #79baff;
--success: #43c17a;
--warning: #f3ad4e;
--danger: #f36b6b;
--radius-control: 0.5rem;
--radius-card: 0.75rem;
--radius-dialog: 1rem;
--duration-fast: 140ms;
--duration-normal: 220ms;
```

Componentes del design system

Componente	Uso
<code>AppButton</code> / <code>AppIconButton</code> / <code>AppLinkButton</code>	Acciones primarias, secundarias y de solo ícono
<code>AppCard</code>	Contenedor base con borde y superficie

Componente	Uso
<code>AppDialog</code> / <code>AppDrawer</code>	Modal y cajón lateral, ambos con <code>focus trap</code> propio (<code>useFocusTrap.js</code>)
<code>AppInput</code> / <code>AppSelect</code> / <code>AppCheckbox</code> / <code>AppTextarea</code>	Controles de formulario, con estado de error
<code>AppBadge</code>	Estados (pendiente, aprobado, vencido...) en píldora de color
<code>AppAlert</code>	Mensajes de contexto (info, éxito, advertencia, error)
<code>AppEmptyState</code> / <code>AppErrorState</code>	Estados vacíos y de error, con acción de reintento
<code>AppSkeleton</code> / <code>AppSpinner</code>	Estados de carga
<code>AppToast</code>	Notificaciones flotantes temporales

Mobile-first, de verdad

- Rango probado: 360 px hasta pantallas anchas de escritorio.
- Navegación inferior fija (`SocioBottomNav` , `PublicBottomNav`) en pantallas angostas, reemplazando la barra superior.
- El mapa público usa un **bottom sheet de tres posiciones** (cerrado, medio, completo) en móvil, arrastrable con gestos táctiles (`pointerdown` / `pointermove` / `pointerup`), en vez de un panel lateral fijo como en escritorio.
- Los formularios de administración colapsan de grillas de varias columnas a una sola columna por debajo de los `47.99rem` .

Animación

Dos herramientas distintas, elegidas según la necesidad puntual:

- **CSS puro** (`@keyframes` , `stroke-dasharray`) para animaciones de una sola vez al montar un componente — el gráfico de tendencia de peso del socio se "dibuja" de izquierda a derecha y sus puntos aparecen escalonados.
- [motion-v](#) (el puerto de Motion/Framer Motion para Vue) para animación basada en física en la campana de notificaciones: un resorte real (`spring`) le da inercia al ícono y hace que "suene" con un impulso proporcional a cuántas notificaciones nuevas llegaron.

Ambos casos concretos (campana de notificaciones y gráfico de peso) respetan `prefers-reduced-motion` : si el sistema operativo tiene animaciones reducidas activadas, esos dos componentes muestran directamente su estado final sin transición. No es una garantía verificada para cada transición CSS menor del resto del sitio (hover, `chevron` que rota al abrir un `<details>` , etc.), que son transiciones cortas y no deberían causar malestar, pero no están explícitamente auditadas una por una.

Accesibilidad

- Botones de solo ícono llevan `aria-label`.
- Diálogos y cajones atrapan el foco (`inert` en el contenido de fondo) y lo devuelven al elemento que los abrió al cerrarse.
- Suite de Playwright con `@axe-core/playwright` corriendo contra las páginas públicas (`frontend/e2e/`).

Nomenclatura visual vs. nomenclatura de código

Los nombres de componentes de UI (`AppButton` , `AppDialog` , `AppCard`) se mantienen en inglés deliberadamente — son vocabulario de diseño de interfaz estándar en cualquier equipo, en cualquier idioma. En cambio, los componentes que representan **conceptos del negocio** de GymTrack sí están en español: `SocioTopNav` , `SocioPanel` , `NotificacionBell` , `SelectorContextoGimnasio` . El criterio completo está en [Decisiones Técnicas](#).

Ver también: [Arquitectura del Sistema](#) · [Decisiones Técnicas](#) · [Análisis y Requerimientos](#)

Decisiones Técnicas

← [Mapa de Contenidos](#) · Ver también [Bitácora del Proyecto](#) · [Arquitectura del Sistema](#)

Registro de decisiones de ingeniería no obvias tomadas durante el desarrollo, con el porqué de cada una. No es un changelog de features — para eso está [Bitácora del Proyecto](#).

Renombrar el código de inglés a español (91 archivos)

Decisión: traducir los nombres de dominio de negocio en archivos, clases y stores —

`PaymentController` → `PagoController`, `useMemberStore` → `useSocioStore`, `NotificationBell.vue` → `NotificacionBell.vue`, etc. — dejando sin traducir los sufijos de arquitectura (`Controller`, `Service`, `Repository`, `View`, `Store`) y prefijos de alcance (`Admin`, `Auth`, `Public`, `System`).

Por qué: son términos técnicos universales, no vocabulario del negocio; traducirlos (`ControladorAdmin`) no aporta claridad y sí rompe la convención con la que cualquier desarrollador lee código Vue/PHP en cualquier idioma.

Cómo se hizo, con cuidado: el autoloader de PHP exige que el nombre del archivo coincida con el de la clase (ver [Arquitectura del Sistema](#)), así que cada rename fue: `git mv` del archivo + reemplazo del nombre de la clase/función en **todo** el proyecto (no solo donde se usaba a simple vista), seguido de `php -l`, la suite de Vitest y un `build` de producción para verificar que no quedó ninguna referencia rota. Se hizo en lotes (backend → vistas → stores → componentes) para poder revertir un lote sin perder los anteriores si algo fallaba.

Lo que salió mal en el camino: cuatro archivos (los sub-componentes de la campana animada) nunca habían llegado a `git add`, así que `git mv` los rechazó silenciosamente mientras el reemplazo de texto sí corría sobre los archivos que los importaban — quedaron importaciones apuntando a un nombre en español que todavía no existía en disco. Se detectó con `grep` de verificación posterior y se corrigió con un `mv` simple. Lección: verificar el estado real en disco después de un rename masivo, no asumir que `git mv` tuvo éxito solo porque el script siguió corriendo.

Rate limiting de login: por cuenta, no solo por IP

Problema real: el limitador de intentos (`RateLimiter`) bloqueaba por combinación (acción, cuenta, IP). Un atacante que rota de IP entre intentos nunca acumula 5 intentos fallidos en ninguna IP individual — el límite nunca se activa.

Decisión: agregar un modo `$global=true` que usa una IP "comodín" fija en vez de la real, así todos los intentos de una misma cuenta (sin importar la IP) caen en una sola fila de la tabla `rate_limit_attempts`. Se llama a ambos modos (por IP y global) en paralelo en cada intento de login.

Por qué esta forma y no una migración de esquema: la clave única de la tabla ya es `(accion, sujeto_hash, ip_hash)` — usar un valor constante como "IP" para el modo global reutiliza esa misma clave sin tocar el esquema ni escribir una consulta de agregación nueva.

Aislamiento de tenant verificado dos veces (defensa en profundidad)

Se encontró que `AdminController::crearMembresia / crearClase` buscaban al socio/instructor con un método sin filtro de gimnasio (`buscarPorId`), aunque el `INSERT` posterior sí validaba el tenant con un `EXISTS`. No era explotable, pero permitía filtrar (por diferencia de respuesta 404) si un ID de usuario existía en otro gimnasio. Se corrigió usando el mismo método con aislamiento (`buscarPorIdEnDataset`) que ya usaba el resto del controlador — la corrección fue reutilizar un método existente, no escribir uno nuevo.

Placeholders de gimnasios demo: sacar el texto, no parchear el CSS

Síntoma reportado: el nombre del gimnasio se veía cortado en la miniatura del mapa.

Primer intento (insuficiente): cambiar `object-fit: cover` a `contain` en un solo lugar. Resolvía el síntoma ahí, pero el mismo problema existía en cualquier otro lugar de la app donde esa misma imagen se mostrara recortada (la tarjeta de la lista, por ejemplo).

Causa real: las 5 imágenes SVG de demostración tenían el nombre del gimnasio dibujado como texto dentro de la propia imagen, pegado al borde. Ese texto además era redundante: el nombre y el badge "Demostración" ya se muestran como texto real al lado de cada tarjeta.

Decisión final: sacar el texto de las 5 imágenes y dejar solo el fondo abstracto — así cualquier recorte, en cualquier proporción, queda seguro. Se revirtió el CSS a `cover` porque ya no hay nada crítico que proteger de un recorte.

Sin dependencia nueva para animaciones simples, sí para animación física

Para el gráfico de tendencia de peso (una animación de "dibujado" de una sola vez) se usó CSS puro (`stroke-dasharray` con el truco de `pathLength="1"`) — no había necesidad de una librería para algo que el navegador ya resuelve. Para la campana de notificaciones, que necesita que el ícono reaccione con inercia real a la velocidad de su propio movimiento (un resorte físico, no una transición lineal), se agregó `motion-v` como dependencia — no había forma razonable de replicar eso a mano sin reimplementar un motor de física.

Base de datos: solo hasta donde el archivo lo permite

Al escribir [Modelo de Base de Datos](#) se verificó cada tabla contra `grep "CREATE TABLE"` en vez de listar de memoria — un primer borrador tenía un error de conteo (49 en vez de 50 tablas reales) y una tabla completa (`actividades`) faltante. Se corrigió antes de dejarlo como documentación permanente.

Mencionado acá como recordatorio: **cualquier número o nombre en esta documentación que hable de código debe salir de una verificación directa, no de memoria.**

Ver también: [Bitácora del Proyecto](#) · [Arquitectura del Sistema](#) · [Seguridad de la API](#)

Bitácora del Proyecto

← [Mapa de Contenidos](#) · Ver también [Decisiones Técnicas](#) · [Metodología y Alcance](#)

Esta bitácora mezcla dos fuentes distintas, marcadas por separado para no confundirlas:

1. Una **etapa temprana estimada** (abril–julio 2026), reconstruida a pedido del equipo porque el repositorio actual no la registra — el trabajo arrancó en una carpeta anterior que no llegó hasta acá.
2. La **línea de tiempo real**, verificada directamente con `git log --all --pretty=format:'%ad | %s' --date=format:'%Y-%m-%d'` sobre este repositorio, desde agosto de 2026 en adelante.

Etapa temprana, estimada (abril–julio 2026) — a confirmar

El equipo confirmó que arrancó en **abril de 2026**, después de las vacaciones, en una carpeta/repositorio anterior cuya historia de Git no se trasladó a este. Lo que sigue es una **reconstrucción razonable**, no una fila de hechos verificados — está pendiente que el equipo pase los hitos reales para reemplazar cada entrada.

Período (estimado)	Qué probablemente pasó
Abril 2026	Formación del equipo y elección del tema (plataforma de descubrimiento y gestión de gimnasios)
Abril–mayo 2026 (estimado)	Relevamiento de requerimientos, casos de uso y decisión de stack (Vue 3 en el frontend, PHP sin framework en el backend, MySQL)
Mayo 2026 (estimado)	Primer diseño del modelo de base de datos, antes de escribir las migraciones que hoy están en <code>database/migrations/</code>
Mayo–junio 2026 (estimado)	Definición de identidad visual y sistema de diseño (ver Sistema de Diseño)
Junio–julio 2026 (estimado)	Configuración del entorno de desarrollo (Docker, esqueleto del proyecto) en la carpeta anterior
~Julio–agosto 2026	El trabajo se traslada a la carpeta/repositorio actual — a partir de acá, todo queda verificado por Git en la sección de abajo

 **Esta tabla es una interpretación pedida explícitamente por el equipo para llenar un hueco real, no un hecho verificado en ningún archivo.** Reemplazar cada fila apenas se confirmen fechas y hitos concretos — avisar para actualizarla.

Línea de tiempo real (commits)

Fecha	Commit
2026-08-05	<code>chore:</code> establecer línea base aprobada de la fase 1

Fecha	Commit
2026-08-05	feat: completar la experiencia pública (fase 2)
2026-08-05	feat: agregar roles demo y contexto de gimnasio (fase 3)
2026-08-06	docs: registrar candidatos de limpieza sin borrar archivos
2026-08-06	chore: conservar archivos sin uso para revisión final
2026-08-06	feat: completar identidad segura y roles (fase 3)
2026-08-06	chore: conservar línea base aprobada de la fase 3
2026-08-06	feat: completar el shell de administración (fase 4)
2026-08-07	feat: completar operación multi-tenant (fase 5)
2026-08-07	docs: consolidar la documentación del proyecto
2026-08-07	chore: ordenar el repositorio público
2026-08-12	feat: completar reservas de clases y asistencia
2026-08-12	feat: completar pagos, finanzas y reportes
2026-08-12	feat: completar mapa y geolocalización
2026-08-12	fix: priorizar gimnasios en el explorador del mapa
2026-08-12	fix: cerrar brechas de autorización de tenant
2026-09-04	release: publicar versión de septiembre

Lectura de esta línea de tiempo: casi todo el desarrollo funcional ocurrió en dos ventanas cortas e intensas (5–7 de agosto, y el 12 de agosto), no de forma pareja en el tiempo — algo común en un proyecto de cierre de materia con entregas concentradas. El release de septiembre marca el cierre formal antes de la etapa de endurecimiento descrita abajo.

Sesión de endurecimiento y pulido (septiembre 2026, sin commits propios todavía)

Trabajo realizado después del release de septiembre, documentado acá porque todavía no fue confirmado en un commit separado:

- **Auditoría de seguridad** end-to-end (backend, frontend, infraestructura) — ver hallazgos aplicados en [Seguridad de la API](#) y [Decisiones Técnicas](#).
- **Incidente real de infraestructura:** el contenedor de MySQL perdió su conexión a la red interna de Docker; al reconectarlo apareció un segundo problema (puerto 3306 ocupado por otro proceso del host). Resuelto deshabilitando temporalmente la publicación de ese puerto en `docker-compose.yml` — el backend igual llega a la base por la red interna, así que no afectó ninguna funcionalidad.

- **Rate limiting de login reforzado** (por cuenta además de por IP).
- **Corrección de aislamiento de tenant** en `AdminController` (búsqueda de socio/instructor sin filtro de gimnasio).
- **Renombrado completo del código a español** (91 archivos entre backend y frontend), ver [Decisiones Técnicas](#).
- **Mejoras visuales del área del socio**: campana de notificaciones animada (`motion-v`), gráfico de tendencia de peso rediseñado y animado, corrección del recorte de texto en las imágenes de demostración, corrección de la posición del ícono "siguiente" en las tarjetas del panel.
- **Corrección de un memory leak** en el store de recursos de administración (`adminResources.js`): cada búsqueda agregaba una función de reseteo nueva en vez de reemplazar la anterior.
- **Resets de sesión incompletos**: varios stores de Pinia no se limpiaban al cerrar sesión, dejando datos del usuario anterior en memoria hasta la siguiente recarga completa.

Cómo mantener esta bitácora al día

Cada vez que se cierre una etapa de trabajo real, agregar una fila a la tabla de arriba (si hay un commit) o una entrada nueva a la sección de la sesión en curso (si todavía no se confirmó). Evitar reconstruir fechas de memoria — usar siempre `git log` como fuente de verdad.

Ver también: [Decisiones Técnicas](#) · [Metodología y Alcance](#) · [Conclusiones del Proyecto](#)

Estrategia de Testing

← [Mapa de Contenidos](#) · Ver también [Seguridad de la API](#)

Cuatro capas de prueba reales en el repositorio, verificadas por conteo directo de archivos (no de memoria).

1. Unitarios de frontend — Vitest

14 archivos de test, 38 tests, corriendo con `npm run test:run` (verificado en esta sesión: 14 passed / 38 passed).

```
frontend/src/components/admin/__tests__/AdminDataTable.spec.js
frontend/src/components/notifications/__tests__/NotificacionBell.spec.js
frontend/src/components/public/__tests__/PublicHeader.spec.js
frontend/src/components/system/__tests__/DemoDataNotice.spec.js
frontend/src/components/ui/__tests__/AppButton.spec.js
frontend/src/components/ui/__tests__/AppCheckbox.spec.js
frontend/src/components/ui/__tests__/AppDialog.spec.js
frontend/src/components/ui/__tests__/AppInput.spec.js
frontend/src/stores/__tests__/gimnasiosPublicos.spec.js
frontend/src/stores/__tests__/notificaciones.spec.js
frontend/src/stores/__tests__/pagos.spec.js
frontend/src/stores/__tests__/promociones.spec.js
frontend/src/stores/__tests__/socio.spec.js
frontend/src/utils/__tests__/geo.spec.js
```

Cobertura real, sin inflar: de los 12 stores de Pinia del proyecto, solo 5 tienen test (`socio` , `pagos` , `promociones` , `notificaciones` , `gimnasiosPublicos`). **`auth.js` — el store que maneja sesión y CSRF, el más sensible de todos — no tiene test propio.** Es una brecha real, no un detalle menor.

2. End-to-end — Playwright

8 specs en `frontend/e2e/` : `admin.spec.js` , `auth.spec.js` , `demo-roles.spec.js` , `map-visual.spec.js` , `payments.spec.js` , `public.spec.js` , `schedule.spec.js` , `tenant-operations.spec.js` . Incluyen verificación de accesibilidad con `@axe-core/playwright` .

Advertencia real sobre estos tests: varios specs (por ejemplo `payments.spec.js`) hacen `test.skip(!password, ...)` cuando la variable de entorno `DEMO_USER_PASSWORD` no está definida. Correr la suite sin exportar esa variable da un resultado de "0 failed" que en realidad es "todo saltado" — un falso verde si no se presta atención.

3. Contratos de API — scripts de shell y PHP

20 archivos en `tests/` , dos formatos distintos:

- **Scripts `.sh`** (`tenant_isolation.sh` , `schedule_booking_attendance.sh` , `payments_finance_reports.sh` , `map_geolocation.sh` , `promociones_notificaciones.sh` , `miembro_http.sh` , `tenant_operations.sh` , `demo_dataset.sh` , `auth_flows.sh` , `admin_api.sh` , `api_smoke.sh` , `csrf_http_status.sh`): pegan contra una instancia real corriendo en Docker y verifican comportamiento end-to-end.
- **Scripts `.php`** (`identity_contracts.php` , `integrity_contracts.php` , `contratos_miembro.php` , `contratos_frontend_miembro.php` , `contratos_calendario_promociones_notificaciones.php` , `production_infrastructure_contracts.php` , `security_tenant_contracts.php` , `security_http_status_contracts.php` , `security_concurrency_contracts.php`): algunos ejecutan código real y verifican el resultado; **otros (los de seguridad de tenant y de infraestructura de producción) verifican con expresiones regulares que cierto texto exista en el código fuente** (por ejemplo, que un controlador contenga la palabra `requireGym`), no que la lógica funcione. Ese tipo de contrato pasaría igual aunque la función tuviera un bug interno.

4. Integración continua — GitHub Actions

Un único workflow, `.github/workflows/ci.yml` . Lo que corre ahí, verificado: `php -l` sobre todo el backend, `sh -n` sobre los scripts de shell, y los contratos `.php` . **Los scripts `.sh` (que son los que de verdad ejercitan tenant isolation, pagos y agenda contra una base real) no corren en CI** — solo se pueden correr a mano, localmente, contra un `docker compose up` .

Qué significa esto en conjunto

La suite de tests existe y es genuina — no es una carpeta vacía ni tests triviales que no prueban nada. Pero tiene tres brechas concretas que vale la pena nombrar en la defensa en vez de ocultar:

1. El store de autenticación no tiene test unitario propio.
2. Los contratos de seguridad más importantes (aislamiento de tenant) dependen de correrlos manualmente, no de CI.
3. Algunos contratos de "seguridad" verifican texto en el código fuente, no comportamiento en ejecución.

Nombrar estas brechas con precisión (en vez de decir "tenemos 80% de cobertura" sin poder sostenerlo) es más defendible frente a un tribunal que pregunte por el detalle.

Ver también: [Seguridad de la API](#) · [Decisiones Técnicas](#) · [Manual de Despliegue](#)

Seguridad de la API

← [Mapa de Contenidos](#) · Ver también [Arquitectura del Sistema](#) · [Modelo de Base de Datos](#)

A diferencia de un changelog de "vulnerabilidad encontrada y arreglada" (que es lo típico en este tipo de documento), acá la mayoría de las protecciones de GymTrack **ya estaban bien implementadas** antes de la auditoría de esta sesión — el objetivo es documentar el estado real, no dramatizar hallazgos.

Lo que ya funcionaba bien (verificado, no asumido)

- **Sin inyección SQL:** no se encontró ninguna consulta con texto de usuario concatenado directamente; todo pasa por sentencias preparadas de PDO. El único lugar con un nombre de tabla dinámico (`TokenService`) lo valida contra una lista cerrada de tablas permitidas antes de interpolarlo.
- **Aislamiento de tenant real:** casi toda consulta de `models/` filtra por `gimnasio_id`, incluso en capas donde el controlador ya validó el contexto — defensa en profundidad, no un único punto de falla.
- **CSRF con un único punto de verificación:** `AuthMiddleware::verificarSesion()` llama a `SessionManager::verifyCsrf()` en cada `POST / PUT / PATCH / DELETE`; no se encontró ninguna ruta protegida que lo sortee.
- **Sesiones:** `session_regenerate_id(true)` en login y al rotar de contexto, revocación respaldada en base de datos (no solo cookie), expiración por inactividad y absoluta ambas verificadas en el servidor.
- **Tokens de un solo uso** (verificación de correo, recuperación de contraseña): generados con `random_bytes`, guardados como hash SHA-256, consumidos dentro de una transacción con `FOR UPDATE` para que no se puedan usar dos veces en una carrera.
- **Archivos subidos:** tipo MIME verificado con `finfo` del lado del servidor (no solo la extensión), nombre de archivo aleatorio, guardados fuera del webroot, servidos únicamente a través de un controlador autenticado que revalida el tenant.
- **Webhook de Mercado Pago:** firma HMAC-SHA256 verificada con `hash_equals` (comparación segura contra *timing attacks*), y el estado del pago se **confirma directamente con la API de Mercado Pago** después de validar la firma — nunca se aprueba un pago solo porque el navegador volvió a una URL de "éxito".

Lo que se corrigió en esta sesión

Rate limiting de login: por cuenta, no solo por IP

El problema: el limitador de intentos frenaba por combinación (acción, cuenta, IP). Un atacante que distribuye los intentos entre varias IP nunca acumula 5 intentos fallidos en ninguna IP puntual — el bloqueo nunca se activa contra ese patrón.

La corrección: `RateLimiter` ahora acepta un modo `$global=true` que agrupa todos los intentos de una cuenta en una sola fila sin importar la IP de origen. El login llama a ambos modos en paralelo. Ver el detalle técnico en [Decisiones Técnicas](#).

Búsqueda sin aislamiento de tenant en `AdminController`

`crearMembresia()` y `crearClase()` buscaban al socio/instructor con `Usuario::buscarPorId()`, sin filtrar por gimnasio, solo para devolver un 404 si no existía — el `INSERT` real sí validaba el tenant después. No era explotable para escribir datos cruzados entre gimnasios, pero sí filtraba (por la diferencia entre 404 y éxito) si un ID de usuario existía en otro gimnasio. Corregido usando el mismo método con aislamiento (`buscarPorIdEnDataset`) que ya usaba el resto del controlador.

Brechas conocidas, documentadas sin maquillar

Estas quedaron identificadas y **no se corrigieron** en esta sesión — se documentan acá para que la defensa las pueda nombrar en vez de que las encuentre el tribunal primero:

- **Turnstile (anti-bot) solo se exige en `APP_ENV=production`**. Un entorno de staging o demo mal configurado queda sin protección de bot en el registro, sin que nada lo avise.
- **Dos formatos de respuesta de API conviviendo.** Controladores nuevos usan `ApiResponse` (`{ok, error, data, meta, request_id}`); controladores más viejos (`AuthController`, `AdminController`, `ReservasController`) arman la respuesta a mano con una forma distinta. Es deuda de mantenibilidad, no un agujero de seguridad, pero obliga al frontend a manejar dos formas de leer un error.
- **Login sin verificación anti-bot (Turnstile solo está en registro).** El rate limiting por cuenta mitiga el credential stuffing, pero no reemplaza a un CAPTCHA tras varios intentos fallidos.
- **Los contratos automatizados de "seguridad de tenant" en `tests/` verifican, en parte, que cierto texto exista en el código fuente (por ejemplo, la palabra `requireGym`), no que la lógica se ejecute correctamente.** Ver el detalle en [Estrategia de Testing](#).

Otras protecciones documentadas en el código

- Cookies de sesión `HttpOnly` siempre, `Secure` cuando `SESSION_SECURE=true` (obligatorio en producción, verificado por `scripts/production-preflight.sh`).
- Auditoría con `request_id` por petición, propagado en las respuestas de error para poder correlacionar un reclamo con un log concreto.
- Consentimiento versionado (`user_consent`s) y baja publicitaria mediante token de un solo uso (`marketing_unsubscribe_tokens`) — nadie puede darse de baja de la lista de otra persona adivinando una URL.
- Carné digital firmado con vencimiento corto — un QR filtrado o reenviado deja de servir en minutos, y no contiene datos personales dentro del código, solo un identificador firmado.

Ver también: [Arquitectura del Sistema](#) · [Modelo de Base de Datos](#) · [Estrategia de Testing](#) · [Decisiones Técnicas](#) · [Implicancias Éticas](#)

Manual de Despliegue

← [Mapa de Contenidos](#) · Ver también [Arquitectura del Sistema](#) · [Seguridad de la API](#)

Desarrollo local

Requisitos: Docker con Docker Compose y Git. Nada más — ni PHP, ni Node, ni MySQL instalados en la máquina.

```
git clone <repo>
cd gymtrack-final1
cp .env.example .env
docker compose up -d --build
```

`docker-compose.yml` define tres servicios reales sobre una red interna (`gymtrack_net`):

Servicio	Imagen	Puerto expuesto	Qué hace al arrancar
db	mysql:8.0	(ninguno actualmente — ver nota abajo)	Aplica el esquema base + las 10 migraciones montadas en <code>docker-entrypoint-initdb.d/</code>
backend	php:8.2-apache + pdo_mysql / mbstring	8080:80	<code>composer install</code> sin dependencias de desarrollo, crea la carpeta de subida de archivos, arranca Apache
frontend	node:20-alpine	5173:5173	<code>npm ci + vite --host 0.0.0.0</code>

Nota real sobre el puerto de MySQL: actualmente `docker-compose.yml` tiene comentada la publicación `3306:3306` porque, en esta máquina, otro proceso ya ocupaba ese puerto y le impedía a Docker levantar el contenedor. El backend llega a la base de datos igual, por la red interna (`DB_HOST=db`) — lo único que se pierde es poder conectar un cliente MySQL de escritorio directo al puerto del host. Si se necesita ese acceso, hay que liberar el puerto 3306 en el host y descomentar esa línea.

Variables de entorno relevantes (`.env`)

Extracto real de lo que `docker-compose.yml` lee, con sus valores por defecto de desarrollo:

```
APP_ENV=development
DB_DATABASE=gymtrack
DB_USERNAME=gymtrack_user
DB_PASSWORD=gymtrack_pass
```

```
SESSION_SECURE=false
MAIL_TRANSPORT=log
PAYMENT_PROVIDER=mercado_pago
PAYMENT_MODE=test
GEOCODING_ENABLED=true
SEED_DEMO_DATA=false
FEATURE_WHATSAPP=false
FEATURE_GOOGLE_CALENDAR_SYNC=false
FEATURE_ADVANCED_ANALYTICS=false
FEATURE_PERSONAL_RECOMMENDATIONS=false
```

Cargar el dataset de demostración

```
APP_ENV=demo
SEED_DEMO_DATA=true
DEMO_USER_PASSWORD=elegí-una-clave-local
```

```
docker compose up -d --force-recreate backend
docker compose exec -T backend php console.php seed:demo
```

Crea 4 cuentas demo (socio, empleado, dueño, administrador general), todas con

`DEMO_USER_PASSWORD` como contraseña. `seed:demo --status` consulta el estado; `seed:demo --remove` borra únicamente los datos demo, sin tocar datos reales.

Cuenta administrativa inicial

```
docker compose exec \
  -e ADMIN_EMAIL=admin@tu-dominio.com \
  -e ADMIN_INITIAL_PASSWORD='una-clave-larga-y-unica' \
  backend php seed_admin.php
```

Producción

El despliegue de producción usa un compose separado (`docker-compose.prod.yml`) con imágenes propias (`backend/Dockerfile.prod` , `frontend/Dockerfile.prod`), contenedores de solo lectura y un servicio adicional, `schema-check` , que **rechaza el arranque del backend si falta alguna migración requerida** en la base de datos — en vez de arrancar silenciosamente con un esquema incompleto.

Pasos reales:

```
chmod 600 .env.production /ruta/privada/msmtprc
scripts/production-preflight.sh .env.production
```

```
docker compose --env-file .env.production -f docker-compose.prod.yml build
docker compose --env-file .env.production -f docker-compose.prod.yml up -d
```

`scripts/production-preflight.sh` bloquea el despliegue si detecta, entre otras cosas: `SEED_DEMO_DATA=true` , `ALLOW_DEMO_DATA_IN_PRODUCTION=true` , cookies no seguras, credenciales de ejemplo sin cambiar, o alguna función beta todavía activada. Ver el detalle de qué chequea exactamente en [Seguridad de la API](#).

El frontend de producción sirve el build estático con Nginx, con un `resolver` de Docker (`127.0.0.11`) configurado explícitamente para el proxy hacia `/api` — sin eso, Nginx cachea la IP del backend al arrancar y un redeploy del backend por sí solo deja al proxy apuntando a una IP vieja.

Correo saliente

En desarrollo, `MAIL_TRANSPORT=log` escribe los correos a un archivo local en vez de enviarlos — se puede inspeccionar el último con:

```
php console.php auth:mail:latest correo@ejemplo.test
```

En producción, `MAIL_TRANSPORT=mail` entrega vía `msmtp` (configurado fuera de Git, con la ruta indicada en `SMTP_CONFIG_FILE`), o `MAIL_TRANSPORT=smtp` para apuntar directo a un proveedor SMTP (Gmail con contraseña de aplicación, por ejemplo, solo para demostraciones).

Mapa y geocodificación

```
GEOCODING_ENABLED=true
GEOCODING_PROVIDER_URL=https://nominatim.openstreetmap.org/search
GEOCODING_CACHE_DAYS=30
```

Proxy PHP propio con caché (`geocodificacion_cache`) y un límite de una consulta externa por segundo — pensado para el volumen de un proyecto académico, no para tráfico de producción real sin cambiar de proveedor.

Ver también: [Arquitectura del Sistema](#) · [Seguridad de la API](#) · [Librerías Utilizadas](#)

Librerías Utilizadas

← [Mapa de Contenidos](#) · Ver también [Arquitectura del Sistema](#)

Contenido exacto de `frontend/package.json` y `backend/composer.json` al momento de escribir esto — no una lista genérica de "lo que suele usar un proyecto Vue".

Frontend (`package.json`)

Dependencias de producción

Paquete	Versión	Para qué se usa en GymTrack
<code>vue</code>	<code>^3.4.21</code>	Framework de UI (Composition API en todo el proyecto)
<code>vue-router</code>	<code>^4.3.0</code>	Ruteo de la SPA, con guards de navegación
<code>pinia</code>	<code>^2.1.7</code>	Estado global: 12 stores, uno por dominio (<code>socio</code> , <code>pagos</code> , <code>agenda</code> , <code>admin</code> ...)
<code>axios</code>	<code>^1.7.9</code>	Cliente HTTP único (<code>services/api.js</code>), con interceptor de errores
<code>leaflet</code>	<code>^1.9.4</code>	Mapa interactivo del catálogo público
<code>@tabler/icons-vue</code>	<code>^3.46.0</code>	Set de íconos usado en toda la app
<code>qrcode</code>	<code>^1.5.4</code>	Generación del QR del carné digital del socio
<code>motion-v</code>	<code>^2.4.2</code>	Animación con física real (resorte) de la campana de notificaciones
<code>@vueuse/core</code>	<code>^14.4.0</code>	Peer dependency de <code>motion-v</code>
<code>@fontsource-variable/manrope</code>	<code>^5.3.0</code>	Tipografía Manrope autoalojada (sin depender de Google Fonts en runtime)

Dependencias de desarrollo

Paquete	Versión	Para qué
<code>vite</code>	<code>^8.0.13</code>	Bundler y servidor de desarrollo
<code>@vitejs/plugin-vue</code>	<code>^6.0.2</code>	Soporte de Vue para Vite
<code>vitest</code>	<code>^4.1.10</code>	Test runner unitario
<code>@vue/test-utils</code>	<code>^2.4.11</code>	Montaje de componentes en los tests
<code>jsdom</code>	<code>^29.1.1</code>	DOM simulado para Vitest

Paquete	Versión	Para qué
<code>@playwright/test</code>	<code>^1.62.1</code>	Tests end-to-end
<code>@axe-core/playwright</code> / <code>axe-core</code>	<code>^4.12.1</code>	Auditoría de accesibilidad automatizada dentro de los tests e2e

Backend (`composer.json`)

```
{
  "require": {
    "php": "^8.2",
    "phpmailer/phpmailer": "^7.0"
  }
}
```

Sí, es así de corto — **una sola dependencia externa real**. No hay Laravel, no hay Symfony, no hay un ORM: el resto del backend (routing, autorización, PDO, PDF/Excel, HMAC, QR firmado) está escrito a mano sobre la biblioteca estándar de PHP. `phpmailer/phpmailer` se usa solo para armar y enviar el correo HTML de verificación/recuperación en producción (en desarrollo, `MAIL_TRANSPORT=log` escribe el correo a un archivo local en vez de enviarlo).

El campo `"license": "proprietary"` en `composer.json` es la única declaración de licencia que existe en el repositorio hoy — no hay un archivo `LICENSE` en la raíz (ver [Metodología y Alcance](#)).

Comandos reales

```
# Backend: instalar dependencias
cd backend && composer install

# Frontend: instalar dependencias
cd frontend && npm install

# Frontend: servidor de desarrollo (Vite, con proxy a la API)
npm run dev

# Frontend: build de producción
npm run build

# Frontend: tests unitarios
npm run test:run

# Frontend: tests end-to-end
npm run test:e2e
```

En Docker no hace falta correr estos comandos a mano: `docker compose up` instala las dependencias de ambos lados como parte del arranque de los contenedores (ver [Manual de Despliegue](#)).

Ver también: [Arquitectura del Sistema](#) · [Manual de Despliegue](#) · [Sistema de Diseño](#)

Implicancias Éticas

← [Mapa de Contenidos](#) · Ver también [Seguridad de la API](#)

Este documento describe decisiones de diseño que tienen una dimensión ética explícita, verificadas contra el comportamiento real del sistema — no es una lista de buenas intenciones sin respaldo en el código.

Privacidad de la ubicación

El mapa público pide la ubicación del navegador solo con consentimiento explícito del usuario, y la distancia a cada gimnasio **se calcula en el propio navegador** (`frontend/src/utils/geo.js`). El backend nunca recibe ni almacena las coordenadas del visitante — no hay una tabla ni un campo donde esa ubicación quede guardada. Rechazar el permiso no bloquea el uso del mapa: simplemente no se puede ordenar por cercanía.

Salud: el IMC como referencia, nunca como diagnóstico

El cálculo de IMC en el área del socio se presenta siempre junto a una advertencia explícita de que es orientativo y no constituye un diagnóstico médico. Es una decisión deliberada frente a la tentación de presentar un número de salud con más autoridad de la que realmente tiene — GymTrack no es una aplicación médica y no debería sonar como una.

Datos de demostración, claramente marcados

Todo registro de demostración lleva una marca (`is_demo` , `demo_dataset_id`) visible tanto en la base de datos como en la interfaz (badge "Demostración" en cada tarjeta). El dataset completo se puede eliminar con un comando (`seed:demo --remove`) sin tocar datos reales — importante para que, si el sistema alguna vez pasa de demo a uso real, no queden gimnasios ficticios mezclados con gimnasios reales en el catálogo público.

No confirmar acciones que no ocurrieron

Las cuatro funciones beta (WhatsApp, sincronización automática de Google Calendar, analítica avanzada, recomendaciones personalizadas) están desactivadas por defecto y **fallan de forma explícita** cuando no están configuradas, en vez de simular éxito. Por ejemplo, el canal de WhatsApp no configurado no le dice al usuario "mensaje enviado" si en realidad no hay ningún proveedor real detrás. Es una postura ética concreta: la interfaz no debe mentir sobre lo que el sistema realmente hizo.

Pagos: verificación real, no confianza en el cliente

Un pago con Mercado Pago nunca se aprueba porque el navegador volvió a una URL de "éxito" — el estado se confirma siempre contra la API del proveedor, después de validar la firma del webhook (ver [Seguridad de la API](#)). Esto protege tanto al gimnasio (contra un pago falso) como al socio (contra que se le cobre sin que el pago realmente haya ocurrido).

Consentimiento de notificaciones

Las preferencias de notificación y el consentimiento de marketing están versionados (`user_consent`) y la baja se hace con un token de un solo uso — nadie puede darse de baja de la lista de otra persona adivinando una URL, y el sistema puede demostrar en qué momento y bajo qué versión de política una persona dio su consentimiento.

Aislamiento de datos entre gimnasios

Cada gimnasio es un tenant aislado (ver [Arquitectura del Sistema](#)); un empleado de un gimnasio no puede ver datos de socios de otro gimnasio en el que no trabaja. Es tanto una decisión técnica como una decisión ética: los datos personales de un socio (teléfono, historial de pagos, mediciones corporales) no deberían ser visibles para alguien sin una relación operativa real con esa persona.

Accesibilidad como requisito, no como extra

El proyecto corre auditorías automatizadas de accesibilidad (`@axe-core/playwright`) sobre las páginas públicas como parte de su suite de pruebas — no es una promesa sin verificación, es un chequeo que corre. Ver el alcance real (qué páginas cubre y cuáles no) en [Estrategia de Testing](#).

Lo que este documento no dice

No afirma que GymTrack cumple con una normativa específica (GDPR, LGPD, etc.) de forma certificada — eso requeriría una revisión legal que está fuera del alcance de un proyecto académico. Lo que sí describe son decisiones de diseño concretas y verificables que van en esa dirección.

Ver también: [Seguridad de la API](#) · [Arquitectura del Sistema](#) · [Conclusiones del Proyecto](#)

Conclusiones del Proyecto

← [Mapa de Contenidos](#)

Nota sobre esta nota: la sección grupal de abajo describe hechos verificables del proyecto (tecnologías usadas, desafíos reales documentados en [Decisiones Técnicas](#) y [Bitácora del Proyecto](#)). La sección de reflexión individual está dejada **como plantilla** — no corresponde que una IA invente aprendizajes, dificultades o aspiraciones personales en nombre de otra persona. Cada integrante debería completar la suya.

Sistema

GymTrack — plataforma de descubrimiento de gimnasios y gestión operativa multi-tenant. **Institución:** Proyecto final de Tecnología Web Aplicada · 2026. **Equipo:** Leandro González · Santiago Cáceres · Máximo Díaz · Emilio Escobar · Magdalena Belmonti · Hilaria Pereira.

Resumen técnico verificable

- Backend en PHP 8.2 puro (sin framework), con autoloader por convención y una sola dependencia externa (`phpmailer/phpmailer`).
- Frontend en Vue 3 + Pinia + Vue Router, con un design system propio de ~20 componentes.
- 50 tablas en MySQL 8, aplicadas en 10 migraciones versionadas.
- Multi-tenant real: aislamiento de datos por gimnasio, permisos independientes por gimnasio para una misma persona.
- Integración con Mercado Pago mediante webhook firmado, sin confiar nunca en la redirección del navegador.
- Suite de pruebas en cuatro capas: unitarios de frontend (Vitest), end-to-end (Playwright), contratos de API (scripts de shell y PHP) y CI (GitHub Actions) — con brechas reales documentadas en [Estrategia de Testing](#), no ocultas.

Desafíos técnicos reales, documentados en el camino

- **Multi-tenant con permisos independientes por gimnasio**, no roles globales fijos — una misma persona puede tener un rol distinto en cada gimnasio donde participa.
- **Integración de pagos con verificación real:** el checkout de Mercado Pago no se resuelve confiando en el navegador, sino verificando la firma del webhook y consultando al proveedor.
- **Un incidente real de infraestructura** durante el cierre del proyecto: el contenedor de base de datos perdió su conexión de red interna, y al reconectarlo apareció un segundo problema (un puerto ocupado por otro proceso del sistema operativo) — documentado con el diagnóstico completo en [Bitácora del Proyecto](#).

- **Un renombrado de 91 archivos** de inglés a español manteniendo el proyecto funcionando en cada paso, verificado con lint, tests y build después de cada lote — ver [Decisiones Técnicas](#).

Reflexión grupal

(Espacio para que el equipo complete, en conjunto, qué se llevan del proyecto — sin frases genéricas de relleno; lo que realmente les haya pasado.)

Reflexiones individuales

(Cada integrante completa la suya. Sugerencia de estructura, no obligatoria: qué hiciste concretamente, qué aprendiste que no sabías antes de empezar, qué harías distinto si empezaras de nuevo.)

Leandro González

(completar)

Santiago Cáceres

(completar)

Máximo Díaz

(completar)

Emilio Escobar

(completar)

Magdalena Belmonti

(completar)

Hiliana Pereira

(completar)

Ver también: [Bitácora del Proyecto](#) · [Decisiones Técnicas](#) · [Metodología y Alcance](#)

Guion de Presentación

← [Mapa de Contenidos](#)

Estructura sugerida para la defensa oral. No es un guion palabra por palabra — cada punto enlaza a la nota donde está el detalle real para prepararlo.

1. Apertura (1–2 min)

Qué es GymTrack en una frase: una plataforma que conecta el descubrimiento de gimnasios con la gestión diaria del negocio, para cuatro roles distintos (socio, empleado, dueño, administrador general). Ver [Metodología y Alcance](#) para el alcance completo y qué se dejó explícitamente afuera.

2. Demo en vivo (5–8 min)

Sugerido, en este orden, usando las cuentas del dataset demo (ver [Manual de Despliegue](#)):

1. Catálogo público + mapa (como visitante anónimo).
2. Login como socio → panel, reserva de una clase, carné digital con QR.
3. Login como dueño/empleado → cambiar de gimnasio activo, ver socios, registrar un pago manual, ver finanzas.
4. (Opcional, si el tribunal pregunta) modo soporte de un administrador general entrando a un gimnasio ajeno con motivo auditado.

3. Arquitectura (3–4 min)

Un diagrama simple: Vue → API PHP → MySQL, remarcando que **no hay framework backend** y por qué (ver [Arquitectura del Sistema](#)). Mencionar el multi-tenant por `gimnasio_id` como el concepto arquitectónico central del proyecto.

4. Seguridad (2–3 min)

Los 3–4 puntos más fuertes primero (PDO parametrizado, aislamiento de tenant verificado en capas, webhook de pago verificado contra el proveedor real, rate limiting por cuenta), y **una** brecha conocida nombrada con seguridad, no escondida (ver [Seguridad de la API](#)). Nombrar una debilidad real genera más confianza que afirmar que todo es perfecto.

5. Testing y calidad (1–2 min)

Las cuatro capas reales (unitarios, e2e, contratos, CI), con la cifra exacta (14 archivos / 38 tests de Vitest) y **la brecha honesta** de que el store de autenticación no tiene test propio. Ver [Estrategia de Testing](#).

6. Cierre (1 min)

Qué se haría distinto con más tiempo (ver [Preguntas Frecuentes de Defensa](#)), y agradecimiento.

Antes de entrar a la sala

- ☐ Revisar [Guía de Defensa - Backend](#), [Guía de Defensa - Frontend](#) y [Guía de Defensa - Base de Datos](#) la noche anterior, no el mismo día.
- ☐ Levantar el entorno local (`docker compose up -d`) y probar el login de cada rol demo **antes** de la presentación, no durante.
- ☐ Tener a mano las credenciales demo y saber de memoria en qué archivo `.env` están las variables clave (`DEMO_USER_PASSWORD` , `MERCADO_PAGO_ACCESS_TOKEN` , feature flags).
- ☐ Decidir de antemano quién responde qué área (backend / frontend / base de datos / seguridad) según lo que cada integrante realmente hizo.

Ver también: [Preguntas Frecuentes de Defensa](#) · [Guía de Defensa - Backend](#) · [Guía de Defensa - Frontend](#) · [Guía de Defensa - Base de Datos](#)

Guía de Defensa — Backend

← [Mapa de Contenidos](#) · Detalle completo en [Arquitectura del Sistema](#) y [Seguridad de la API](#)

Puntos que hay que poder explicar sin leer, con la razón detrás de cada uno.

1. "¿Por qué no usaron un framework?"

Fue una decisión curricular: PHP 8.2 puro, con un autoloader propio (`spl_autoload_register` en `backend/index.php`) que resuelve una clase buscándola en `config/` , `controllers/` , `models/` , `middleware/` o `services/` por su nombre exacto. Esto obliga a entender (y poder explicar) cosas que un framework normalmente resuelve solo: routing, sesiones, CSRF, autorización.

Riesgo si preguntan más hondo: que pidan mostrar en vivo cómo el autoloader encuentra una clase. Tenerlo fresco: `backend/index.php` , función `spl_autoload_register` .

2. "¿Cómo evitan que un dueño vea datos de otro gimnasio?"

Cada gimnasio es un tenant. La tabla `usuario_gimnasio_rol` conecta usuario + gimnasio + rol; casi toda consulta de `models/` filtra explícitamente por `gimnasio_id` , no solo el controlador. Poder mostrar un ejemplo concreto: `AdminController::verSocio()` usa

`Usuario::buscarPorIdEnDataset($id, $dataset, $gimnasioId)` , no una búsqueda por ID pelada.

Ejemplo real corregido esta sesión (para mostrar que se entiende el patrón, no solo se repite):

`crearMembresia` / `crearClase` tenían una búsqueda sin ese filtro — corregido, ver [Decisiones Técnicas](#).

3. "¿Cómo manejan las contraseñas y sesiones?"

- `password_hash` / `password_verify` (bcrypt), con re-hash automático si cambia el costo recomendado.
- Cookie de sesión `HttpOnly` , `SameSite` , `Secure` en producción — nunca accesible desde JavaScript.
- CSRF con un token separado del de sesión, verificado en cada mutación (`AuthMiddleware::verificarSesion()` → `SessionManager::verifyCsrft()`).
- `session_regenerate_id(true)` en login, para que un ID de sesión previo a autenticarse no sirva después.

4. "¿Qué pasa si alguien intenta forzar un login por fuerza bruta?"

`RateLimiter` bloquea después de 5 intentos, con espera exponencial hasta 15 minutos, **por cuenta y por IP en paralelo** (esto último se agregó en esta sesión — antes solo era por IP, y rotar de IP lo sorteaba). Ver el detalle en [Seguridad de la API](#).

5. "¿Cómo procesan un pago sin poder confiar en el navegador?"

El checkout redirige a Mercado Pago; cuando el navegador vuelve a la URL de "éxito", **eso no aprueba nada**. La aprobación real ocurre cuando llega un webhook firmado (HMAC-SHA256, verificado con `hash_equals`), y ni siquiera ahí se confía ciegamente en el contenido del webhook: el backend consulta directamente a la API de Mercado Pago el estado real del pago antes de actualizarlo.

6. Si preguntan por debilidades (mejor decirlas antes de que las encuentren)

- Turnstile (anti-bot) solo se exige en producción — un entorno mal configurado queda sin esa protección.
- Dos formatos de respuesta JSON conviven (controladores nuevos vs. viejos) — deuda de mantenibilidad documentada, no corregida todavía.
- Algunos "contratos de seguridad" automatizados verifican texto en el código fuente, no comportamiento real — ver [Estrategia de Testing](#).

Decir esto de entrada, con la corrección ya aplicada donde la hay, se sostiene mejor que negar que existan.

Ver también: [Arquitectura del Sistema](#) · [Seguridad de la API](#) · [Guía de Defensa - Base de Datos](#)

Guía de Defensa — Frontend

← [Mapa de Contenidos](#) · Detalle completo en [Sistema de Diseño](#) y [Arquitectura del Sistema](#)

1. "¿Por qué Pinia y no Vuex u otra cosa?"

Pinia es el gestor de estado oficial recomendado por el propio equipo de Vue desde Vue 3; 12 stores, uno por dominio (`socio` , `pagos` , `agenda` , `admin` , `notificaciones` ...), cada uno con su propio `reset()` para limpiar el estado al cerrar sesión.

Si preguntan por un bug real que hubo acá: varios stores no se limpiaban al cerrar sesión — quedaba en memoria el nombre y los pagos del usuario anterior hasta la siguiente recarga completa de la página. Se corrigió centralizando el reset en `resetAdminStores()` . Es un buen ejemplo de bug real + solución real, más convincente que decir que nunca hubo bugs.

2. "¿Cómo se maneja la sesión sin guardar un token en localStorage?"

No se guarda nada sensible en el cliente. La cookie de sesión es `HttpOnly` (invisible para JavaScript); lo único que el store `auth.js` guarda en memoria es el `csrf_token` que devuelve el login, reenviado en el header `X-CSRF-Token` en cada mutación.

3. "¿Por qué el mapa se ve distinto en celular?"

Bottom sheet de tres posiciones (cerrado/medio/completo), arrastrable con gestos táctiles reales (`pointerdown` / `pointermove` / `pointerup`), en vez de un panel lateral fijo — porque un panel lateral de escritorio no entra en una pantalla de 360 px. Ver [Sistema de Diseño](#).

4. "¿Cómo está armada la campana de notificaciones animada?"

Usa `motion-v` (el puerto de Motion/Framer Motion a Vue) para simular un resorte físico real: cuando llega una notificación nueva, el ícono recibe un impulso proporcional a cuántas llegaron, y el número del badge tiene su propia inercia al cambiar. Se justifica usar una librería acá (y no en el gráfico de peso, que es CSS puro) porque replicar física de resorte a mano es mucho más código que un `@keyframes` .

5. "¿Qué pasa si el usuario tiene 'reducir movimiento' activado en su sistema?"

Tanto la campana como el gráfico de tendencia de peso respetan `prefers-reduced-motion` : si está activado, se muestran directamente en su estado final, sin animar. (Nota honesta si preguntan: esto está verificado para esos dos componentes puntuales, no auditado transición por transición en todo el sitio.)

6. "¿Por qué algunos nombres de archivo están en inglés y otros en español?"

Regla aplicada de forma consistente: **vocabulario del negocio** en español (`SocioTopNav` , `PagoController` del lado del backend, `useSocioStore`), **vocabulario de arquitectura/UI** genérica en inglés (`AppButton` , `Controller` , `Store` , `View`) porque son términos técnicos estándar en cualquier stack, en cualquier idioma. Ver el razonamiento completo en [Decisiones Técnicas](#).

7. Si preguntan por un bug visual real y cómo se encontró

El nombre del gimnasio aparecía cortado en la miniatura del mapa. La causa no era el CSS de esa miniatura puntual, sino que las 5 imágenes de demostración tenían el nombre dibujado como texto dentro del propio SVG — texto que además era redundante porque ya se muestra como texto real al lado. Se sacó el texto de las imágenes en vez de parchear el recorte en cada lugar donde esa imagen se mostrara chica. Buen ejemplo de "arreglar la causa, no el síntoma" — ver [Decisiones Técnicas](#).

Ver también: [Sistema de Diseño](#) · [Arquitectura del Sistema](#) · [Decisiones Técnicas](#)

Guía de Defensa — Base de Datos

← [Mapa de Contenidos](#) · Detalle completo en [Modelo de Base de Datos](#)

1. "¿Cuántas tablas tiene el sistema?"

50, contadas directamente con `grep "CREATE TABLE"` sobre las migraciones reales — no una cifra de memoria. Repartidas en: identidad y seguridad (14), gimnasios y catálogo (3), personas dentro de un gimnasio (5), membresías (3), agenda y asistencia (5), pagos y finanzas (4), promociones y notificaciones (5), experiencia del socio (7), demo/control interno (4).

2. "¿Cómo evolucionó el esquema? ¿Por qué tantas migraciones?"

10 migraciones versionadas, cada una atada a una etapa funcional del proyecto (identidad, roles, administración, operación multi-tenant, agenda, pagos, promociones, experiencia del socio, geocodificación). Cada una (salvo las dos primeras) tiene su reversión (`.down.sql`) — se puede mostrar en vivo con `ls database/migrations/`.

Si preguntan por qué las dos primeras no tienen reversión: son las más fundacionales, crean la base multi-tenant completa; revertirlas de forma segura pedía más cuidado del que da un `DROP` genérico, y quedó documentado como una brecha real, no resuelta — ver [Modelo de Base de Datos](#).

3. "¿Cómo se aísla un gimnasio de otro a nivel de base de datos?"

No hay una base de datos por gimnasio — es una sola base, con `gimnasio_id` como columna de aislamiento en casi toda tabla operativa. El aislamiento se hace a *nivel de fila* (row-level), reforzado en cada consulta de `models/`, no confiado a un único chequeo en el controlador.

4. "¿Por qué esta decisión y no bases separadas por tenant?"

Más simple de operar: una sola migración, un solo backup, una sola conexión. Suficiente para el volumen de un proyecto académico, y demuestra un patrón real de la industria (multi-tenancy por fila) sin la complejidad operativa de gestionar N bases de datos.

5. "¿Qué pasa con los datos sensibles? ¿Todo se guarda en texto plano?"

No. Ejemplos concretos:

- Contraseñas: `password_hash` (bcrypt), nunca en claro.
- Tokens de verificación/recuperación: hash SHA-256, nunca el token real.

- Rate limiting (`rate_limit_attempts`): guarda `sujeto_hash` e `ip_hash` (HMAC), no el correo ni la IP reales.

6. "¿Cómo se relacionan pagos y su historial?"

`pagos` guarda el estado actual; `pago_eventos` guarda **cada cambio de estado**, no solo el final — necesario para poder auditar un pago después del hecho (por ejemplo, si un socio reclama que pagó y el estado dice lo contrario, se puede ver la secuencia completa de eventos, incluyendo la respuesta del webhook de Mercado Pago).

7. "El archivo `gymtrack_database.sql` — ¿ese es el esquema completo?"

No, y es importante decirlo así si preguntan: ese archivo por sí solo describe un esquema de un solo gimnasio, sin `gimnasio_id`. El esquema real es ese archivo **más** las 10 migraciones aplicadas en orden. Leer solo el bootstrap da una idea equivocada del sistema actual.

Ver también: [Modelo de Base de Datos](#) · [Arquitectura del Sistema](#) · [Seguridad de la API](#)

Preguntas Frecuentes de Defensa

← [Mapa de Contenidos](#)

Preguntas transversales que no encajan en una sola guía técnica. Las específicas de cada capa están en [Guía de Defensa - Backend](#), [Guía de Defensa - Frontend](#) y [Guía de Defensa - Base de Datos](#).

"¿Por qué GymTrack y no otro rubro?"

(Completar con la motivación real del equipo — no hay una respuesta genérica válida acá.)

"¿Qué tan real es el dataset de demostración?"

Cinco gimnasios ficticios, marcados explícitamente como demo en la base de datos (`is_demo` , `demo_dataset_id`) y en la interfaz (badge "Demostración"). Se puede crear, resetear o eliminar con `console.php seed:demo` , sin afectar datos reales. No hay usuarios reales ni gimnasios reales operando en el sistema — es un proyecto académico, no un producto en producción.

"¿Está esto en producción? ¿Cuántos usuarios tiene?"

No, y hay que decirlo directo si preguntan: es un proyecto de cierre de materia con un entorno de demostración, no un sistema con usuarios reales. No hay ninguna cifra de uso real que citar.

"¿Qué harían distinto si tuvieran más tiempo?"

Basado en las brechas ya documentadas y no maquilladas:

- Agregar tests unitarios al store de autenticación (`auth.js`), hoy sin cobertura propia — ver [Estrategia de Testing](#).
- Correr los contratos de shell (`tests/*.sh`) en CI, no solo a mano.
- Reemplazar los contratos de "seguridad" basados en buscar texto en el código fuente por pruebas que ejecuten el comportamiento real.
- Unificar el formato de respuesta de la API entre controladores nuevos y viejos.
- Exigir Turnstile también en entornos que no sean estrictamente `production` .

"¿Cuál fue el mayor desafío técnico?"

(Elegir con criterio propio entre los documentados en [Decisiones Técnicas](#) y [Bitácora del Proyecto](#) — por ejemplo el multi-tenant con permisos independientes por gimnasio, o el renombrado de 91 archivos sin romper el autoloader de PHP — y poder explicarlo con el mismo nivel de detalle que está ahí, no de memoria superficial.)

"¿Por qué está todo en español si el código de otros proyectos suele estar en inglés?"

Fue una decisión deliberada de legibilidad para el equipo: los nombres de dominio de negocio (`Socio` , `Pago` , `Agenda`) están en español, mientras que los términos de arquitectura estándar (`Controller` , `Service` , `View`) se dejaron en inglés porque son universales en cualquier stack. Ver [Decisiones Técnicas](#) para el criterio completo, incluido lo que salió mal en el camino (referencias rotas por archivos que nunca habían llegado a Git).

"¿Qué pasa si el webhook de Mercado Pago llega falsificado?"

Se verifica la firma HMAC-SHA256 con `hash_equals` (comparación resistente a *timing attacks*) antes de procesar nada. Si la firma no coincide, se descarta. Incluso con firma válida, el estado del pago no se toma del cuerpo del webhook: se consulta directamente a la API de Mercado Pago.

Ver también: [Guía de Defensa - Backend](#) · [Guía de Defensa - Frontend](#) · [Guía de Defensa - Base de Datos](#) · [Guion de Presentación](#)